

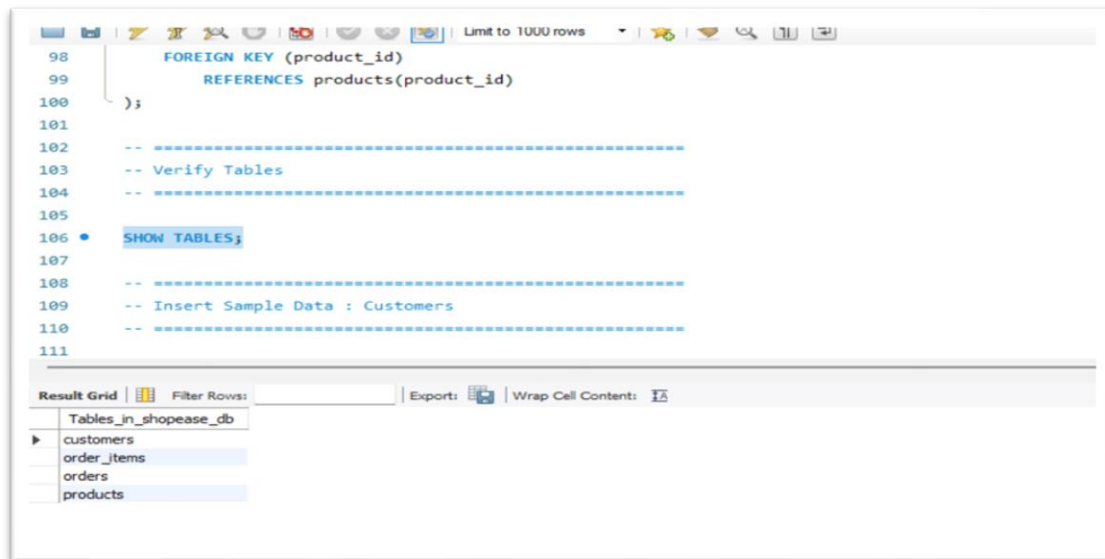
Week 2 Assignment – SQL Queries

(OUTPUTS)

Name : Nikita Sanjay Bansode

College: Sanjivani College of Engineering, Kopergaon

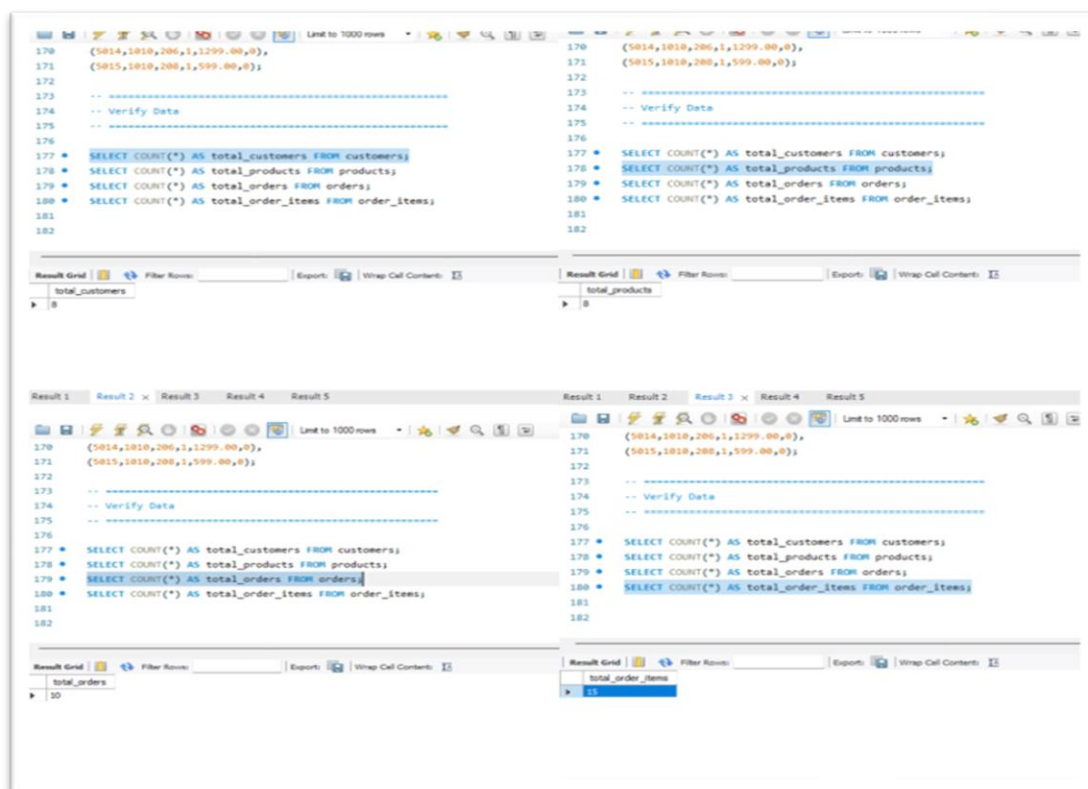
Basics : Creating the database, tables and inserting values in the table.



```
98 FOREIGN KEY (product_id)
99 REFERENCES products(product_id)
100 );
101
102 -- =====
103 -- Verify Tables
104 -- =====
105
106 • SHOW TABLES;
107
108 -- =====
109 -- Insert Sample Data : Customers
110 -- =====
111
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Tables_in_shopease_db
customers
order_items
orders
products



```
170 (5014,1010,200,1,1299.00,0),
171 (5015,1010,200,1,599.00,0));
172
173 -- =====
174 -- Verify Data
175 -- =====
176
177 • SELECT COUNT(*) AS total_customers FROM customers;
178 • SELECT COUNT(*) AS total_products FROM products;
179 • SELECT COUNT(*) AS total_orders FROM orders;
180 • SELECT COUNT(*) AS total_order_items FROM order_items;
181
182
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

total_customers
8

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

total_products
8

Result 1 | Result 2 | Result 3 | Result 4 | Result 5

Result 1 | Result 2 | Result 3 | Result 4 | Result 5

```
170 (5014,1010,200,1,1299.00,0),
171 (5015,1010,200,1,599.00,0));
172
173 -- =====
174 -- Verify Data
175 -- =====
176
177 • SELECT COUNT(*) AS total_customers FROM customers;
178 • SELECT COUNT(*) AS total_products FROM products;
179 • SELECT COUNT(*) AS total_orders FROM orders;
180 • SELECT COUNT(*) AS total_order_items FROM order_items;
181
182
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

total_orders
10

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

total_order_items
15

Section A — SQL Basics (SELECT, Constraints, Primary Keys)

Q1. Write a query to display all columns and rows from the customer's table.

The screenshot shows a SQL IDE window titled "01_Section_A". The query editor contains the following SQL code:

```
-- SECTION A : SQL BASICS
--
-- Q1. Display all records from customers table
SELECT *
FROM customers;
--
-- Q2. Display customer's first name, last name and city
```

The result grid below the query editor displays the following data:

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

The bottom of the window shows tabs for "customers 1", "customers 2", and "products 3".

Q2. Retrieve only the first_name, last_name, and city of all customers.

The screenshot shows a SQL IDE window titled "01_Section_A". The query editor contains the following SQL code:

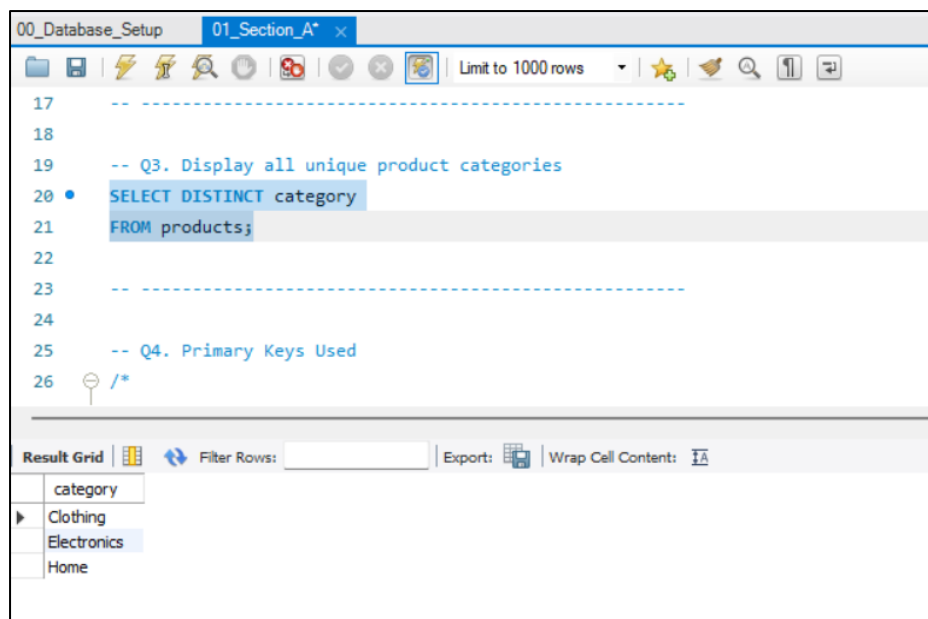
```
SELECT *
FROM customers;
--
-- Q2. Display customer's first name, last name and city
SELECT first_name, last_name, city
FROM customers;
--
```

The result grid below the query editor displays the following data:

first_name	last_name	city
Aarav	Sharma	Mumbai
Priya	Patel	Ahmedabad
Rohan	Gupta	Delhi
Sneha	Reddy	Hyderabad
Vikram	Singh	Jaipur
Ananya	Iyer	Chennai
Karan	Mehta	Pune
Divya	Nair	Kochi

The bottom of the window shows tabs for "customers 1", "customers 2", and "products 3".

Q3. List all unique categories available in the products table.

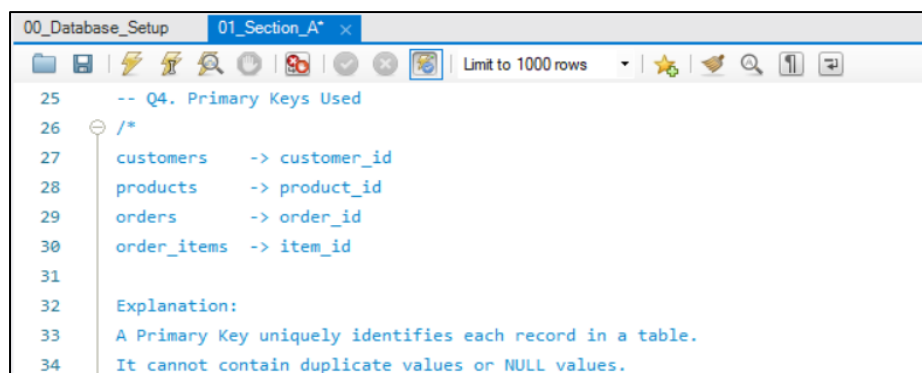


The screenshot shows a database IDE window titled '00_Database_Setup' with a sub-tab '01_Section_A*'. The SQL editor contains the following code:

```
17  -----
18
19  -- Q3. Display all unique product categories
20  SELECT DISTINCT category
21  FROM products;
22
23  -----
24
25  -- Q4. Primary Keys Used
26  /*
```

Below the editor, the 'Result Grid' is visible, showing a table with one column named 'category'. The table contains three rows: 'Clothing', 'Electronics', and 'Home'.

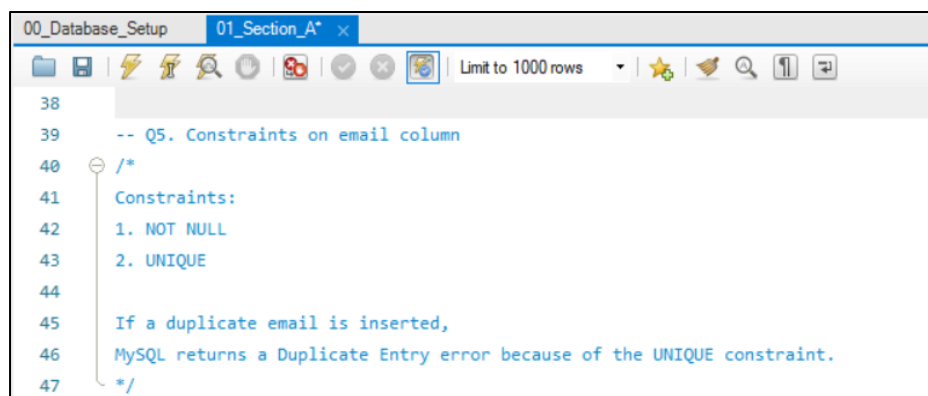
Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.



The screenshot shows a database IDE window titled '00_Database_Setup' with a sub-tab '01_Section_A*'. The SQL editor contains the following code:

```
25  -- Q4. Primary Keys Used
26  /*
27  customers    -> customer_id
28  products     -> product_id
29  orders       -> order_id
30  order_items  -> item_id
31
32  Explanation:
33  A Primary Key uniquely identifies each record in a table.
34  It cannot contain duplicate values or NULL values.
```

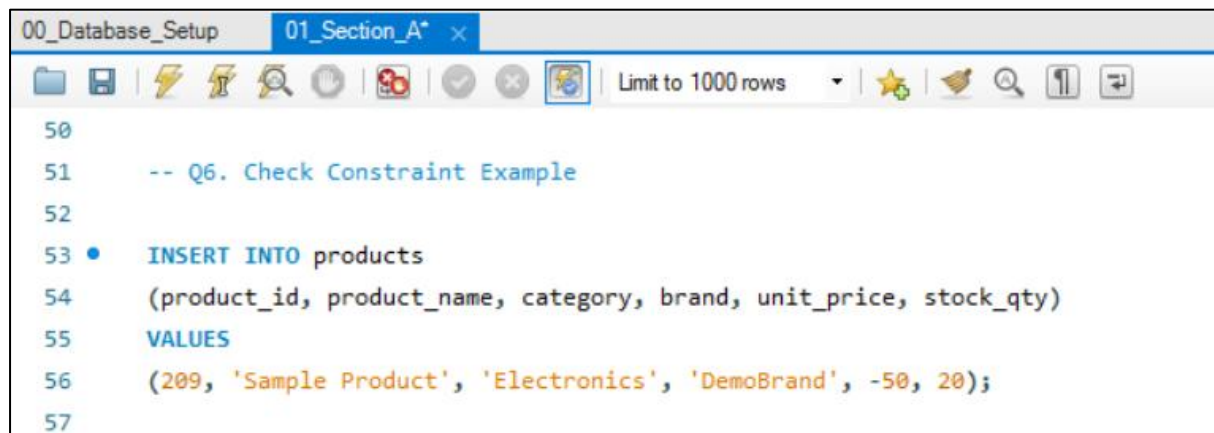
Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?



The screenshot shows a database IDE window titled '00_Database_Setup' with a sub-tab '01_Section_A*'. The SQL editor contains the following code:

```
38
39  -- Q5. Constraints on email column
40  /*
41  Constraints:
42  1. NOT NULL
43  2. UNIQUE
44
45  If a duplicate email is inserted,
46  MySQL returns a Duplicate Entry error because of the UNIQUE constraint.
47  */
```

Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explain the error



The screenshot shows a SQL IDE window with two tabs: '00_Database_Setup' and '01_Section_A*'. The active tab contains a SQL script with the following lines:

```
50
51  -- Q6. Check Constraint Example
52
53  •  INSERT INTO products
54      (product_id, product_name, category, brand, unit_price, stock_qty)
55      VALUES
56      (209, 'Sample Product', 'Electronics', 'DemoBrand', -50, 20);
57
```

Error:

Error Code: 3819. Check constraint 'products_chk_1' is violated.	0.000 sec
OK	0.000 sec
✖ 31 17:17:28 INSERT INTO products (product_id, product_name, category, brand, unit_price, stock_qty) VALUES (209, 'Sa...	
✔ 32 17:31:46 EXPLAIN SELECT DISTINCT category FROM products	

Section B — Filtering & Optimization (WHERE, Indexes)

Q7. Retrieve all orders with status = 'Delivered'.

The screenshot shows a database IDE with a tab labeled '02_Section_B'. The SQL editor contains the following query:

```
-- Q7. Display all delivered orders

SELECT *
FROM orders
WHERE status = 'Delivered';

-----

-- Q8. Electronics products with price greater than 2000
```

The 'Result Grid' at the bottom displays the results of the query. It includes a toolbar with options like 'Filter Rows', 'Edit', 'Export/Import', and 'Wrap Cell Content'. The data is as follows:

	order_id	customer_id	order_date	status	total_amount
▶	1001	101	2024-08-01	Delivered	4498.00
	1002	102	2024-08-03	Delivered	799.00
	1004	101	2024-08-10	Delivered	3499.00
	1006	105	2024-08-15	Delivered	5898.00
	1008	103	2024-08-20	Delivered	899.00
	1010	108	2024-08-28	Delivered	1598.00
*	NULL	NULL	NULL	NULL	NULL

Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.

The screenshot shows the same database IDE with a new SQL query for Q8:

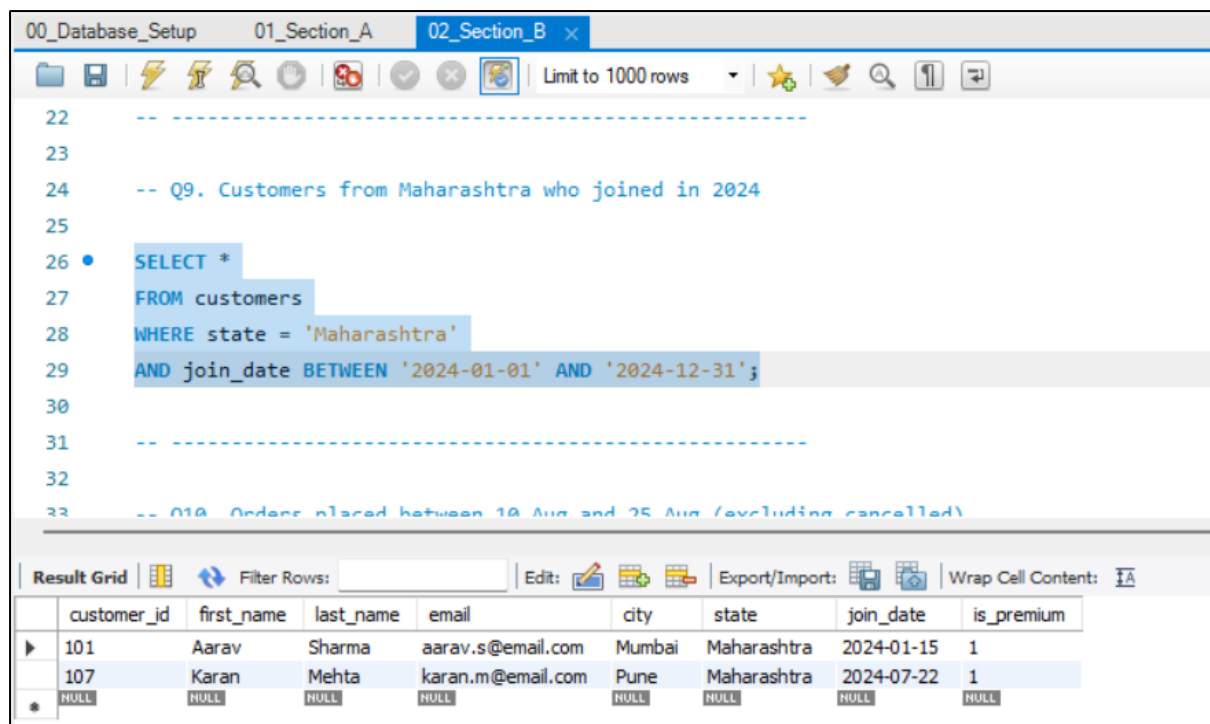
```
-- Q8. Electronics products with price greater than 2000

SELECT *
FROM products
WHERE category = 'Electronics'
AND unit_price > 2000;
```

The 'Result Grid' displays the results of this query. The data is as follows:

	product_id	product_name	category	brand	unit_price	stock_qty
▶	203	Smart Watch	Electronics	Noise	2999.00	150
	205	Bluetooth Speaker	Electronics	JBL	3499.00	200
*	NULL	NULL	NULL	NULL	NULL	NULL

Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.



The screenshot shows a database IDE with a SQL editor and a result grid. The SQL editor contains the following query:

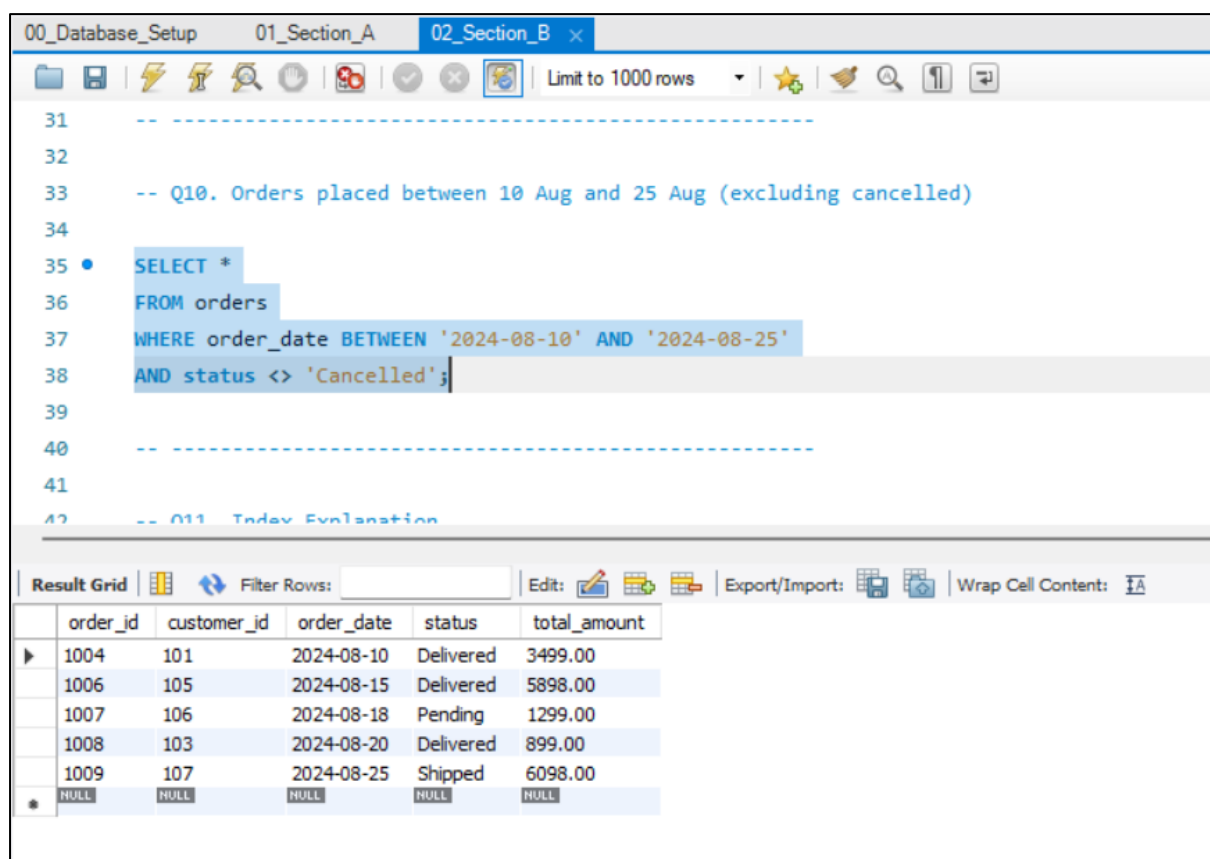
```
-- Q9. Customers from Maharashtra who joined in 2024

SELECT *
FROM customers
WHERE state = 'Maharashtra'
AND join_date BETWEEN '2024-01-01' AND '2024-12-31';
```

The result grid displays the following data:

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.



The screenshot shows a database IDE with a SQL editor and a result grid. The SQL editor contains the following query:

```
-- Q10. Orders placed between 10 Aug and 25 Aug (excluding cancelled)

SELECT *
FROM orders
WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'
AND status <> 'Cancelled';
```

The result grid displays the following data:

order_id	customer_id	order_date	status	total_amount
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1007	106	2024-08-18	Pending	1299.00
1008	103	2024-08-20	Delivered	899.00
1009	107	2024-08-25	Shipped	6098.00
NULL	NULL	NULL	NULL	NULL

Q11. Explain what the index `idx_orders_date` does. How would it improve the performance of a query that filters orders by `order_date`? Write a sample query that would benefit from this index.

```
00_Database_Setup    01_Section_A    02_Section_B x
-- Q11. Index Explanation
/*
The index idx_orders_date is created on the order_date column.

It helps MySQL quickly locate records based on order dates
instead of scanning the entire table.

Sample query using this index:
*/

SELECT *
FROM orders
WHERE order_date = '2024-08-15';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: I

	order_id	customer_id	order_date	status	total_amount
▶	1006	105	2024-08-15	Delivered	5898.00
*	NULL	NULL	NULL	NULL	NULL

Q12. If you run: `SELECT * FROM customers WHERE YEAR(join_date) = 2024;` — would the index on `join_date` be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable)

[illegible]

Section C — Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

Q13. Count the total number of orders in the orders table.

The screenshot shows a SQL IDE window with the following tabs: 00_Database_Setup, 01_Section_A, 02_Section_B, and 03_Section_C (active). The toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following code:

```
6
7  -- Q13. Total number of orders
8
9  • SELECT COUNT(*) AS Total_Orders
10 FROM orders;
11
```

Below the editor, the 'Result Grid' is displayed with the following data:

Total_Orders
10

Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.

The screenshot shows a SQL IDE window with the following tabs: 00_Database_Setup, 01_Section_A, 02_Section_B, and 03_Section_C (active). The toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following code:

```
14  -- Q14. Total revenue from Delivered orders
15
16  • SELECT SUM(total_amount) AS Total_Revenue
17 FROM orders
18 WHERE status = 'Delivered';
19
20 -----
```

Below the editor, the 'Result Grid' is displayed with the following data:

Total_Revenue
17191.00

Q15. Calculate the average unit_price of products in each category.

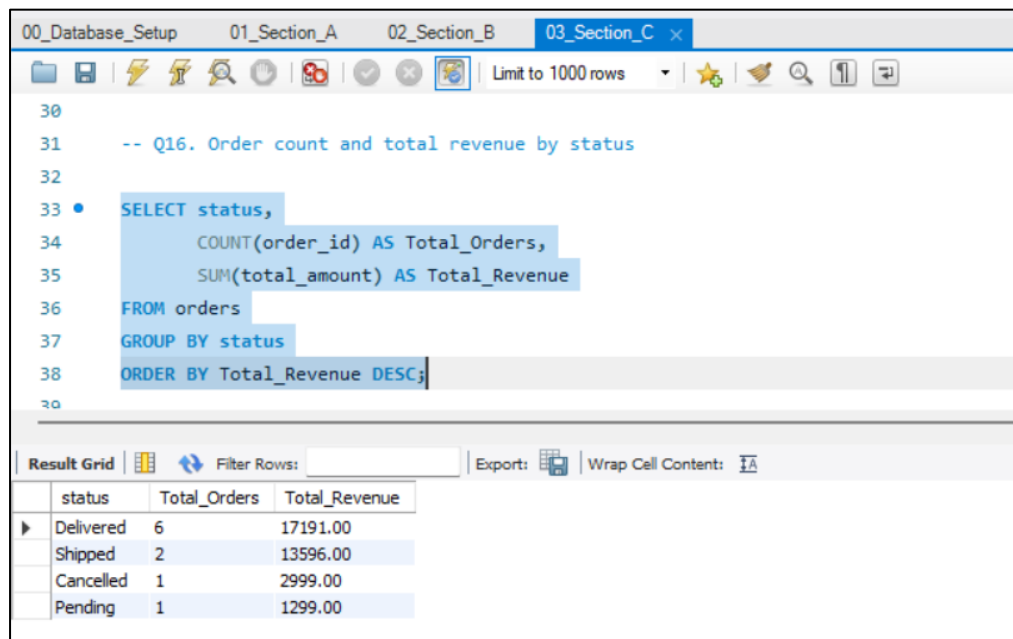
The screenshot shows a SQL IDE window with the following tabs: 00_Database_Setup, 01_Section_A, 02_Section_B, and 03_Section_C (active). The toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following code:

```
22  -- Q15. Average product price in each category
23
24  • SELECT category,
25         ROUND(AVG(unit_price),2) AS Average_Price
26 FROM products
27 GROUP BY category;
28
```

Below the editor, the 'Result Grid' is displayed with the following data:

category	Average_Price
Clothing	2699.00
Electronics	2224.00
Home	949.00

Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.



The screenshot shows a database IDE with a tab labeled '03_Section_C'. The SQL editor contains the following query:

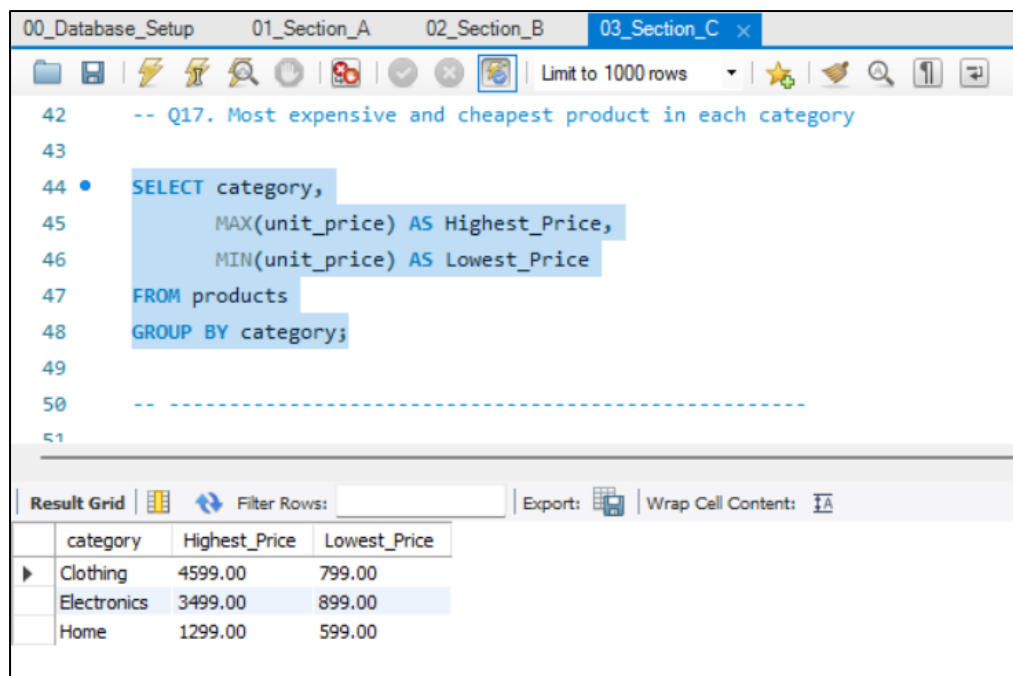
```
-- Q16. Order count and total revenue by status

SELECT status,
       COUNT(order_id) AS Total_Orders,
       SUM(total_amount) AS Total_Revenue
FROM orders
GROUP BY status
ORDER BY Total_Revenue DESC;
```

The results are displayed in a table with the following data:

status	Total_Orders	Total_Revenue
Delivered	6	17191.00
Shipped	2	13596.00
Cancelled	1	2999.00
Pending	1	1299.00

Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.



The screenshot shows a database IDE with a tab labeled '03_Section_C'. The SQL editor contains the following query:

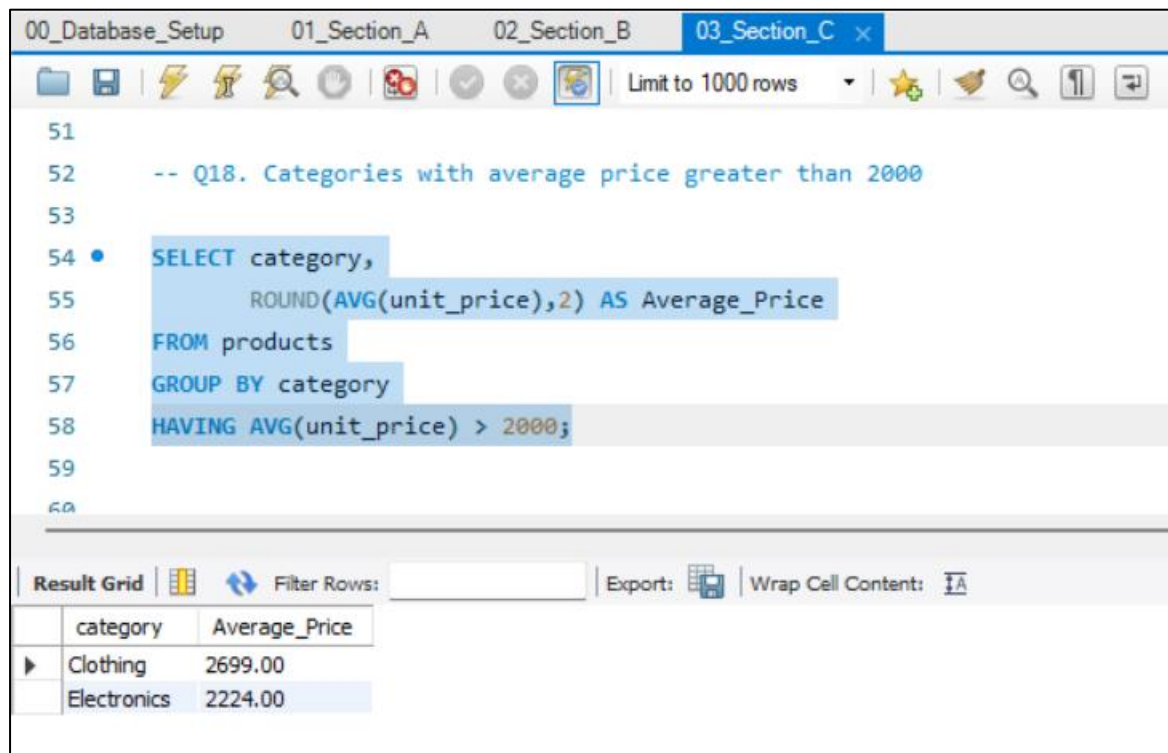
```
-- Q17. Most expensive and cheapest product in each category

SELECT category,
       MAX(unit_price) AS Highest_Price,
       MIN(unit_price) AS Lowest_Price
FROM products
GROUP BY category;
```

The results are displayed in a table with the following data:

category	Highest_Price	Lowest_Price
Clothing	4599.00	799.00
Electronics	3499.00	899.00
Home	1299.00	599.00

Q18. List all product categories where the average unit_price is greater than ₹2000. (Hint: Use HAVING clause)



The screenshot shows a database query editor with a tab labeled '03_Section_C'. The query is as follows:

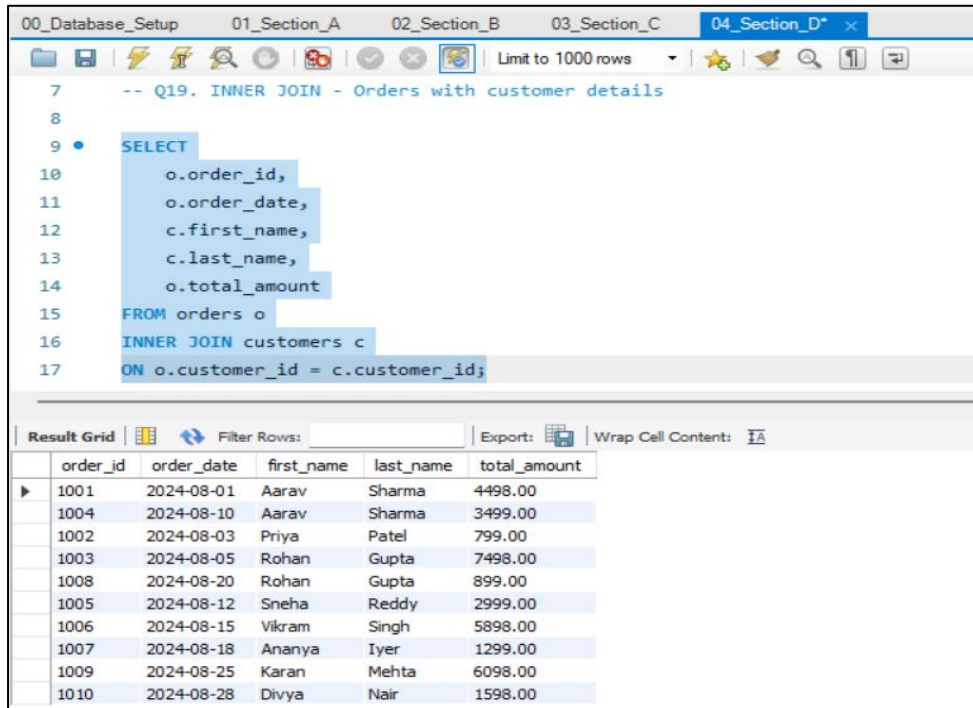
```
51
52  -- Q18. Categories with average price greater than 2000
53
54  • SELECT category,
55      ROUND(AVG(unit_price),2) AS Average_Price
56  FROM products
57  GROUP BY category
58  HAVING AVG(unit_price) > 2000;
59
60
```

Below the query editor, the 'Result Grid' is displayed, showing the results of the query:

	category	Average_Price
▶	Clothing	2699.00
	Electronics	2224.00

Section D — Joins & Relationships

Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name. Show: order_id, order_date, first_name, last_name, total_amount.



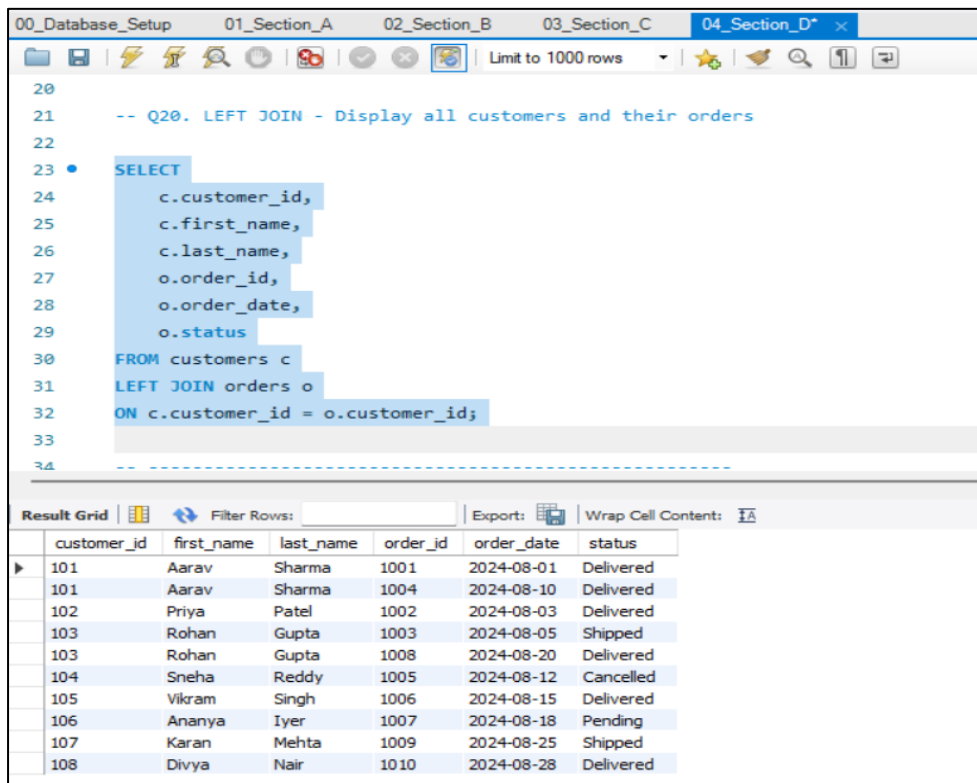
The screenshot shows a database IDE with a query editor and a result grid. The query is an INNER JOIN between the 'orders' and 'customers' tables. The result grid displays 10 rows of data, each representing an order and its corresponding customer details.

```
-- Q19. INNER JOIN - Orders with customer details

SELECT
    o.order_id,
    o.order_date,
    c.first_name,
    c.last_name,
    o.total_amount
FROM orders o
INNER JOIN customers c
ON o.customer_id = c.customer_id;
```

order_id	order_date	first_name	last_name	total_amount
1001	2024-08-01	Aarav	Sharma	4498.00
1004	2024-08-10	Aarav	Sharma	3499.00
1002	2024-08-03	Priya	Patel	799.00
1003	2024-08-05	Rohan	Gupta	7498.00
1008	2024-08-20	Rohan	Gupta	899.00
1005	2024-08-12	Sneha	Reddy	2999.00
1006	2024-08-15	Vikram	Singh	5898.00
1007	2024-08-18	Ananya	Iyer	1299.00
1009	2024-08-25	Karan	Mehta	6098.00
1010	2024-08-28	Divya	Nair	1598.00

Q20. Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for order columns.



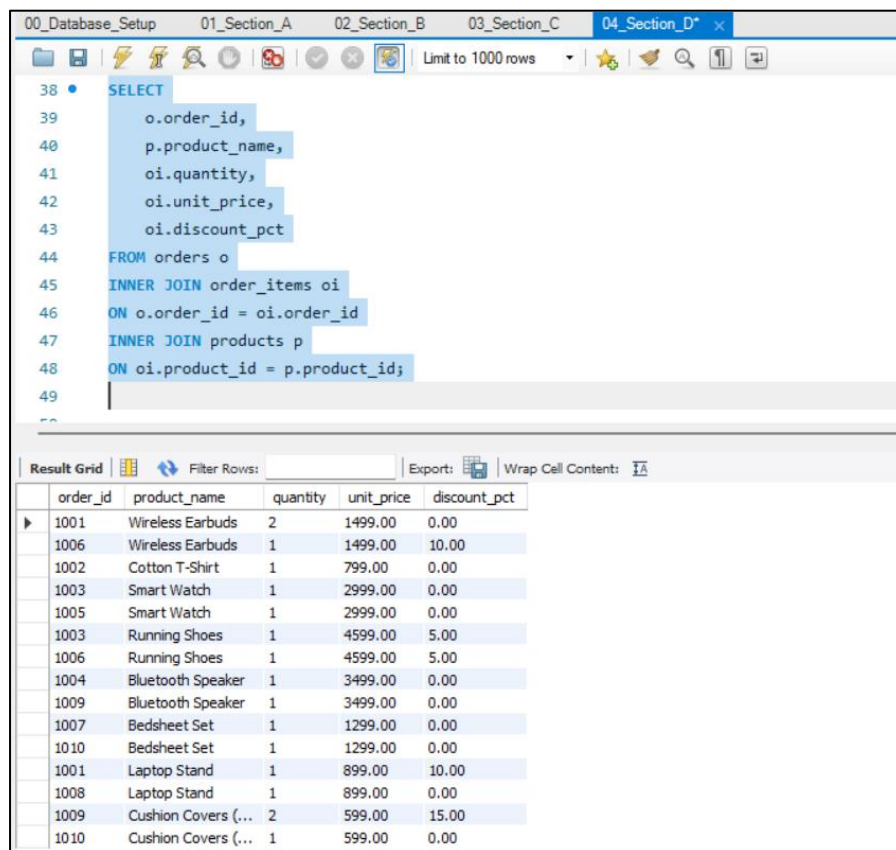
The screenshot shows a database IDE with a query editor and a result grid. The query is a LEFT JOIN between the 'customers' and 'orders' tables. The result grid displays 10 rows of data, each representing a customer and their orders. Customers with no orders have NULL values for the order columns.

```
-- Q20. LEFT JOIN - Display all customers and their orders

SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    o.order_id,
    o.order_date,
    o.status
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;
```

customer_id	first_name	last_name	order_id	order_date	status
101	Aarav	Sharma	1001	2024-08-01	Delivered
101	Aarav	Sharma	1004	2024-08-10	Delivered
102	Priya	Patel	1002	2024-08-03	Delivered
103	Rohan	Gupta	1003	2024-08-05	Shipped
103	Rohan	Gupta	1008	2024-08-20	Delivered
104	Sneha	Reddy	1005	2024-08-12	Cancelled
105	Vikram	Singh	1006	2024-08-15	Delivered
106	Ananya	Iyer	1007	2024-08-18	Pending
107	Karan	Mehta	1009	2024-08-25	Shipped
108	Divya	Nair	1010	2024-08-28	Delivered

Q21. Write a query using JOINS across three tables (orders → order_items → products) to show: order_id, product_name, quantity, unit_price, and discount_pct for each order item.



The screenshot shows a database query editor with a SQL query and its results. The query is as follows:

```

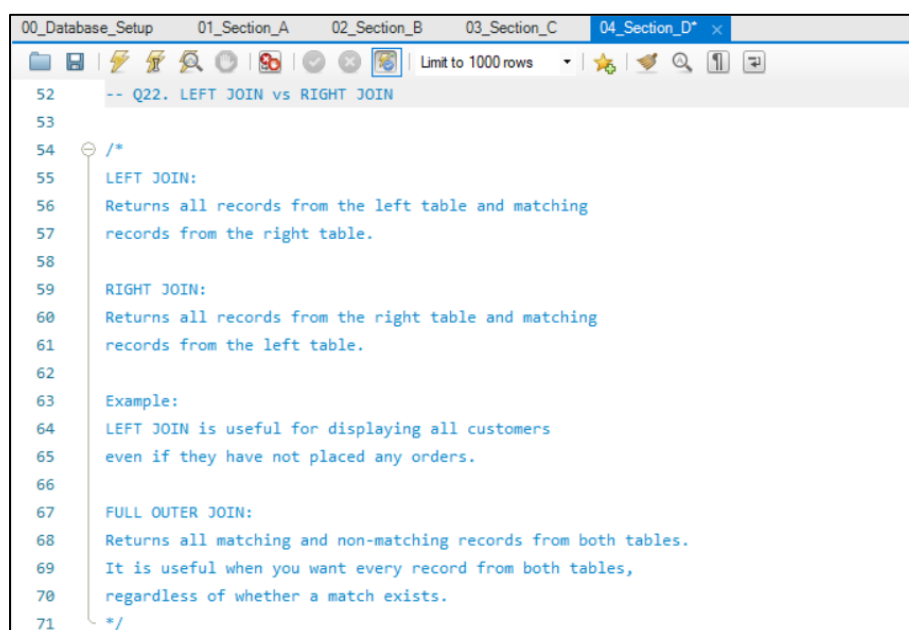
38 SELECT
39     o.order_id,
40     p.product_name,
41     oi.quantity,
42     oi.unit_price,
43     oi.discount_pct
44 FROM orders o
45 INNER JOIN order_items oi
46 ON o.order_id = oi.order_id
47 INNER JOIN products p
48 ON oi.product_id = p.product_id;
49

```

The results are displayed in a table with the following columns: order_id, product_name, quantity, unit_price, and discount_pct. The table contains 15 rows of data.

order_id	product_name	quantity	unit_price	discount_pct
1001	Wireless Earbuds	2	1499.00	0.00
1006	Wireless Earbuds	1	1499.00	10.00
1002	Cotton T-Shirt	1	799.00	0.00
1003	Smart Watch	1	2999.00	0.00
1005	Smart Watch	1	2999.00	0.00
1003	Running Shoes	1	4599.00	5.00
1006	Running Shoes	1	4599.00	5.00
1004	Bluetooth Speaker	1	3499.00	0.00
1009	Bluetooth Speaker	1	3499.00	0.00
1007	Bedsheet Set	1	1299.00	0.00
1010	Bedsheet Set	1	1299.00	0.00
1001	Laptop Stand	1	899.00	10.00
1008	Laptop Stand	1	899.00	0.00
1009	Cushion Covers (...)	2	599.00	15.00
1010	Cushion Covers (...)	1	599.00	0.00

Q22. Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN?



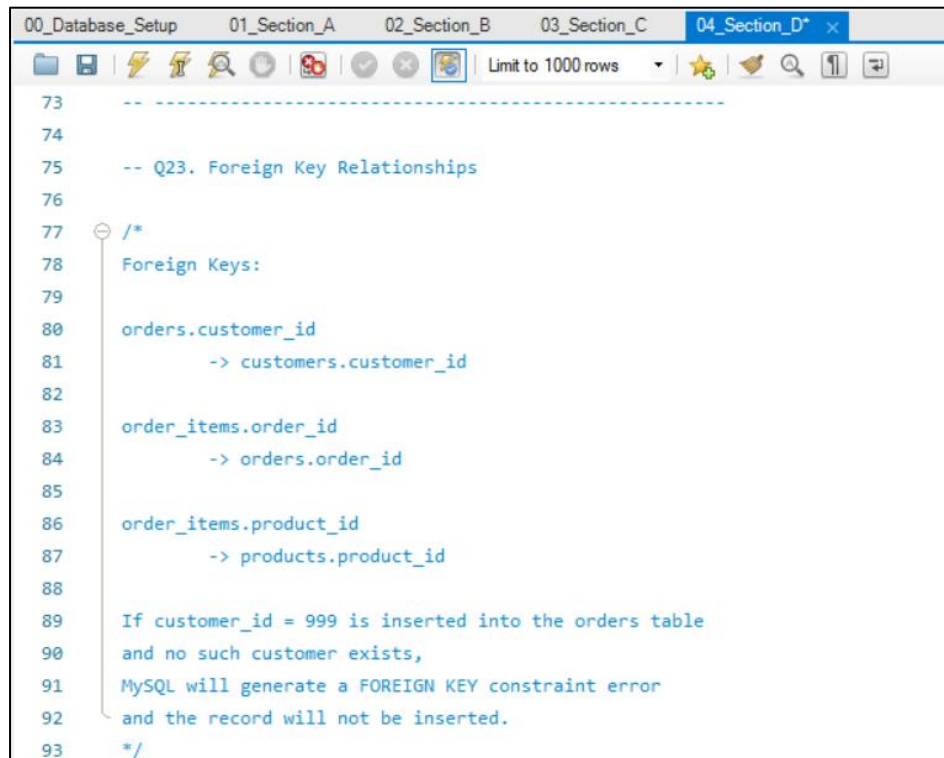
The screenshot shows a database query editor with a SQL query and its results. The query is as follows:

```

52 -- Q22. LEFT JOIN vs RIGHT JOIN
53
54 /*
55 LEFT JOIN:
56 Returns all records from the left table and matching
57 records from the right table.
58
59 RIGHT JOIN:
60 Returns all records from the right table and matching
61 records from the left table.
62
63 Example:
64 LEFT JOIN is useful for displaying all customers
65 even if they have not placed any orders.
66
67 FULL OUTER JOIN:
68 Returns all matching and non-matching records from both tables.
69 It is useful when you want every record from both tables,
70 regardless of whether a match exists.
71 */

```

Q23. Identify all Foreign Key relationships in the schema. Explain what would happen if you tried to insert an order with `customer_id = 999` (which doesn't exist in customers).



```
73  -----
74
75  -- Q23. Foreign Key Relationships
76
77  /*
78  Foreign Keys:
79
80  orders.customer_id
81      -> customers.customer_id
82
83  order_items.order_id
84      -> orders.order_id
85
86  order_items.product_id
87      -> products.product_id
88
89  If customer_id = 999 is inserted into the orders table
90  and no such customer exists,
91  MySQL will generate a FOREIGN KEY constraint error
92  and the record will not be inserted.
93  */
```

Section E — Advanced Concepts (CASE, ACID, Transactions)

Q24. Write a query using CASE to classify products into price tiers:

- 'Budget' → unit_price < 1000
- 'Mid-Range' → unit_price BETWEEN 1000 AND 3000
- 'Premium' → unit_price > 3000 Display: product_name, unit_price, price_tier.

The screenshot shows a database IDE with a SQL query editor and a result grid. The query is as follows:

```
-- Q24. Classify products into price tiers

SELECT
    product_name,
    unit_price,
    CASE
        WHEN unit_price < 1000 THEN 'Budget'
        WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
        ELSE 'Premium'
    END AS price_tier
FROM products;
```

The result grid displays the following data:

product_name	unit_price	price_tier
Wireless Earbuds	1499.00	Mid-Range
Cotton T-Shirt	799.00	Budget
Smart Watch	2999.00	Mid-Range
Running Shoes	4599.00	Premium
Bluetooth Speaker	3499.00	Premium
Bedsheet Set	1299.00	Mid-Range
Laptop Stand	899.00	Budget
Cushion Covers (Set)	599.00	Budget

Q25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered' (all other statuses). Display the result in a single row.

The screenshot shows a database IDE with a SQL query editor and a result grid. The query is as follows:

```
-- Q25. Count Delivered vs Not Delivered orders

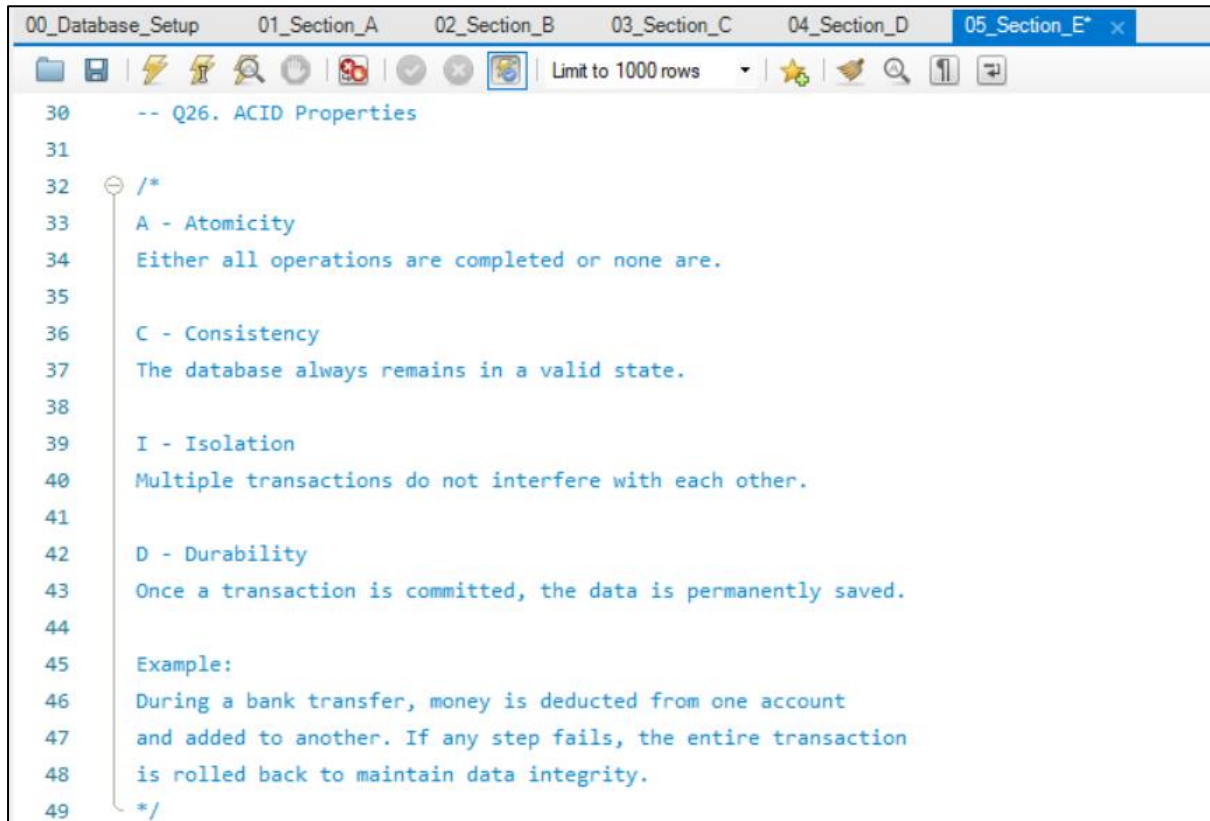
SELECT
    SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS Delivered_Orders,
    SUM(CASE WHEN status <> 'Delivered' THEN 1 ELSE 0 END) AS Not_Delivered_Orders
FROM orders;
```

The result grid displays the following data:

Delivered_Orders	Not_Delivered_Orders
6	4

Q26. Explain each letter of ACID:

- A – Atomicity
- C – Consistency
- I – Isolation
- D – Durability Give a real-world example (e.g., bank transfer) showing why each property is important.

A screenshot of a SQL IDE window with multiple tabs. The active tab is '05_Section_E'. The window contains a SQL comment block explaining the ACID properties. The text is as follows:

```
30  -- Q26. ACID Properties
31
32  /*
33  A - Atomicity
34  Either all operations are completed or none are.
35
36  C - Consistency
37  The database always remains in a valid state.
38
39  I - Isolation
40  Multiple transactions do not interfere with each other.
41
42  D - Durability
43  Once a transaction is committed, the data is permanently saved.
44
45  Example:
46  During a bank transfer, money is deducted from one account
47  and added to another. If any step fails, the entire transaction
48  is rolled back to maintain data integrity.
49  */
```

Q27. Write a SQL transaction that does the following atomically:

1. Insert a new order (order_id=1011, customer_id=102, today's date, 'Pending', 1598.00)
2. Insert two order items for that order
3. Update the stock_qty of the purchased products
4. If any step fails, ROLLBACK the entire transaction. Otherwise, COMMIT. Write the complete BEGIN...COMMIT/ROLLBACK block.


```

53  -- Q27. SQL Transaction
54  •  START TRANSACTION;
55
56  •  INSERT INTO orders
57  VALUES
58  (1011,102,CURDATE(),'Pending',1598.00);
59
60  •  INSERT INTO order_items
61  VALUES
62  (5016,1011,206,1,1299.00,0);
63
64  •  INSERT INTO order_items
65  VALUES
66  (5017,1011,208,1,299.00,0);
67
68  •  UPDATE products
69  SET stock_qty = stock_qty - 1
70  WHERE product_id = 206;
71
72  •  UPDATE products
73  SET stock_qty = stock_qty - 1
74  WHERE product_id = 208;
75
76  •  COMMIT;
77
78  /*
79  If any statement fails before COMMIT,
80  execute:
81  ROLLBACK;

```

✓	55	18:13:47	START TRANSACTION	0 row(s) affected
✓	56	18:13:47	INSERT INTO orders VALUES (1011,102,CURDATE(),'Pending',1598.00)	1 row(s) affected
✓	57	18:13:47	INSERT INTO order_items VALUES (5016,1011,206,1,1299.00,0)	1 row(s) affected
✓	58	18:13:47	INSERT INTO order_items VALUES (5017,1011,208,1,299.00,0)	1 row(s) affected
✓	59	18:13:47	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 206	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✓	60	18:13:47	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 208	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✓	61	18:13:47	COMMIT	0 row(s) affected

Conclusion

This assignment helped me understand the fundamentals of SQL by working with a relational database from start to finish. I learned how to create tables, apply constraints, insert and validate data, perform filtering, aggregation, and joins, and use advanced concepts such as CASE statements and transactions. Completing this assignment strengthened my understanding of SQL queries and improved my confidence in solving database-related problems.